Workpath SDK Device Event Sample Overview

Product Group: HP Workpath SDK
Version: 1.6.3
Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

Table of Contents

1 Introduction

Device Event Sample provides use case scenario how to read event information on a device using DeviceEventsService of the HP Workpath SDK API.

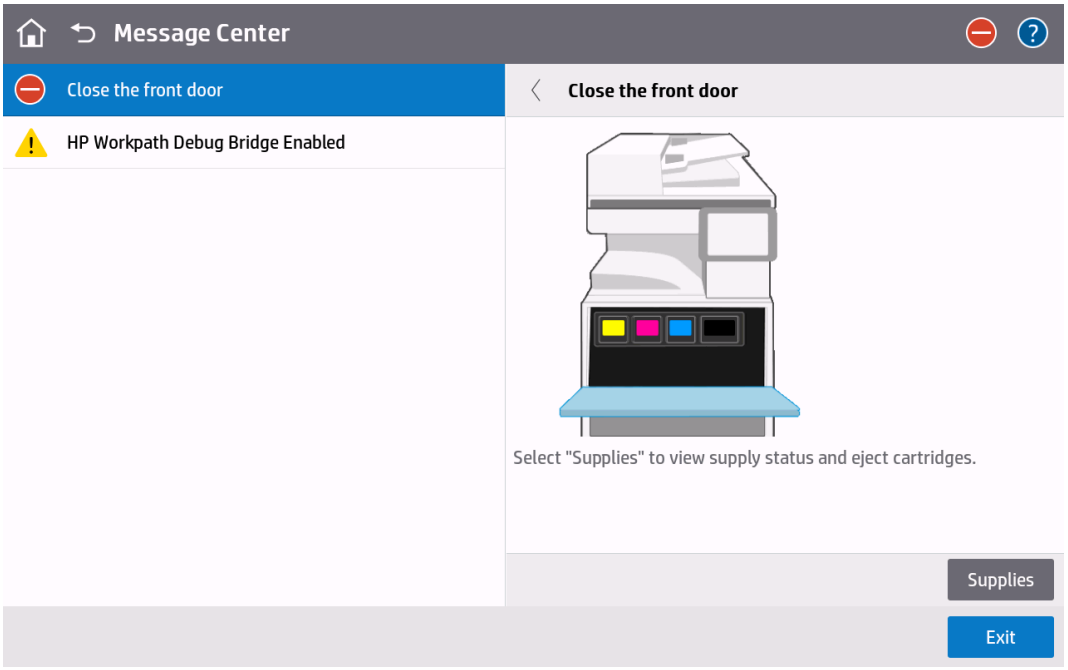
1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

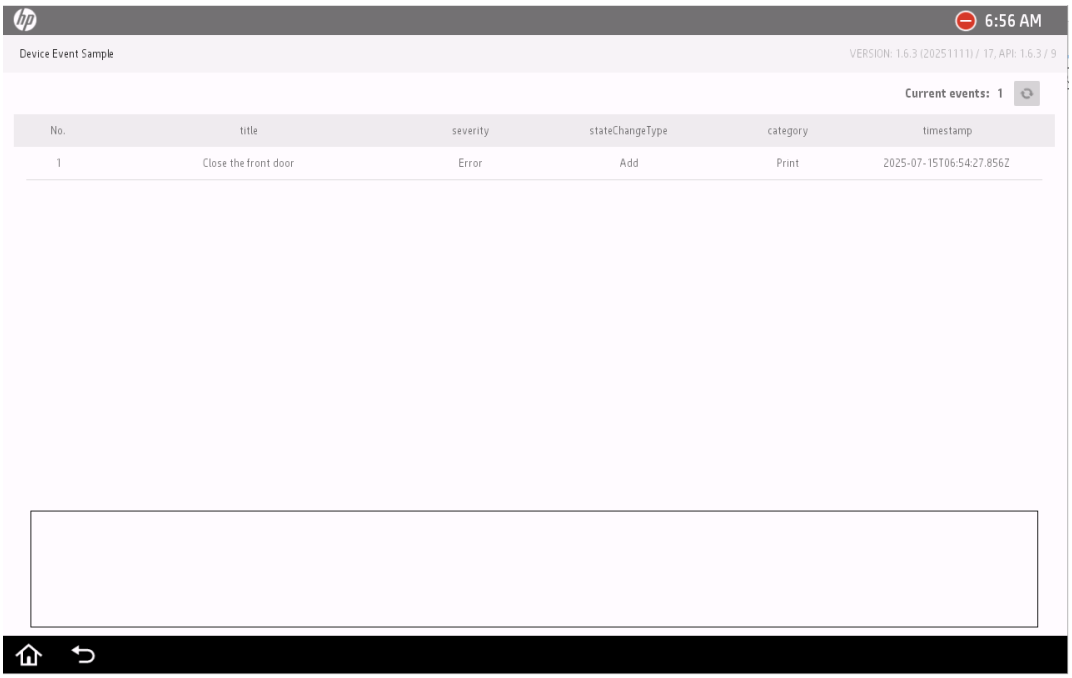
- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- `Name` - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 Device Event Sample on a device

Device Event Sample shows the code and the usage of example how to use DeviceEventsService when event occurs on a device such as “Door open” or others.

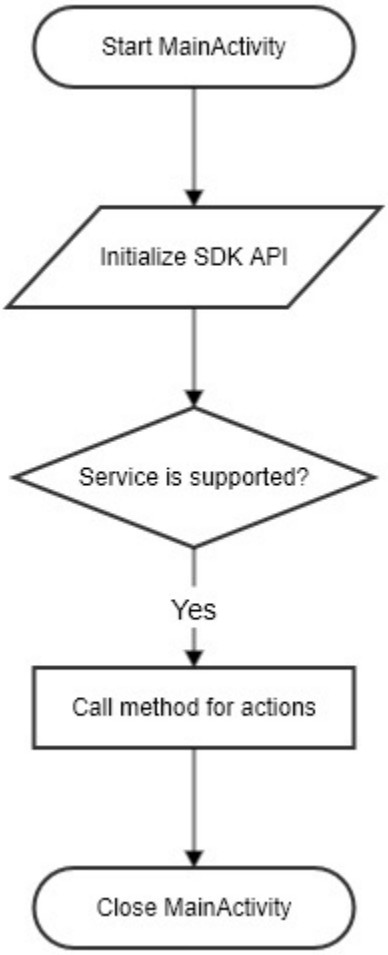


Door Open



A main activity of Device Event Sample

3 Device Event Sample Flow



Flow of Device Event Sample

4 Add permission in manifest

Application must add SDK permission to access new service of the SDK. This shows how to add it for the access of DeviceEventsService.

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="com.hp.workpath.sample.deviceeventsample">
3. <uses-permission android:name="com.hp.workpath.permission.sdk.ACCESS_DEVICE_EVENTS_PERMISSION" />
```

5 Initializing SDK

Application requires to initialize the SDK services when starts with an activity for accessing the SDK services. The `com.hp.workpath.api.Workpath` includes `initialize()` method for this.

The sample shows how to initialize SDK.

1. Call `initialize()` method of Workpath SDK on resume event and clear it on pause event. It is for calling initialization when an activity is switched. Refer to MainActivity.java.

```
1.  @Override
2.      protected void onResume() {
3.          super.onResume();
4.          replaceFragment(mDeviceEventListFragment);
5.          mDeviceEventObserver.register(getApplicationContext());
6.          mInitializationTask = new InitializationTask(this);
7.          mInitializationTask.execute();
8.      }
9.      @Override
10.     protected void onPause() {
11.         super.onPause();
12.         mDeviceEventObserver.unregister(getApplicationContext());
13.         mInitializationTask.cancel(true);
14.         mInitializationTask = null;
15.         if (mAlertDialog != null) {
16.             mAlertDialog.dismiss();
17.             mAlertDialog = null;
18.         }
19.     }
```

Initialization call in Activity Resume

2. Make a task and call `initialize()` method with catching exceptions. If Workpath SDK is not initialized, sdk exception or security exception will be thrown. Refer to InitializationTask.java.

```
1.  @Override
2.      protected InitStatus doInBackground(final Void... params) {
3.          InitStatus status = InitStatus.NO_ERROR;
4.          try {
5.              // initialize Workpath SDK
6.              Workpath.getInstance().initialize(mContextRef.get());
7.              // Check if DeviceEventsService is supported
8.              if (!DeviceEventsService.isSupported(mContextRef.get())) {
9.                  // DeviceEventsService is not supported on this device
10.                 status = InitStatus.NOT_SUPPORTED;
11.             }
12.         } catch (SsdkUnsupportedException sue) {
13.             mThrowable = sue;
14.             status = InitStatus.INIT_EXCEPTION;
15.         } catch (SecurityException se) {
16.             mThrowable = se;
17.             status = InitStatus.INIT_EXCEPTION;
18.         } catch (Throwable t) {
19.             mThrowable = t;
20.             status = InitStatus.INIT_EXCEPTION;
21.         }
22.         return status;
23.     }
```

Exception handling by initializationTask

6 Checking DeviceEventsService Capability

The `isSupported()` method returns whether the DeviceEventsService is supported. Refer to InitializationTask.java.

```
1.    // Check if DeviceEventsService is supported
2.              if (!DeviceEventsService.isSupported(mContextRef.get())) {
3.                  // DeviceEventsService is not supported on this device
4.                  status = InitStatus.NOT_SUPPORTED;
5.              }
```

Available service check by isSupported()

7 Retrieve current events from a device

DeviceEventsService provides methods for retrieving current events on a device.

The code snippet shows example of such queries :

```
1.  @Override
2.      protected List<DeviceEvent> doInBackground(Void... voids) {
3.          try {
4.              return DeviceEventsService.getDeviceEvents(mContextRef.get(), mResult);
5.          } catch (Throwable t) {
6.              mThrowable = t;
7.          }
8.          return null;
9.      }
```

Retrieve current events from a device in DeviceEventTask.java

8 Monitoring an event from a device

When device's event is changed, the Workpath SDK will send an event. The application can detect the change through `onChange()` event. The code snippet shows example of such queries :

1. Register observer to get an event.

```
1.  @Override
2.      protected void onResume() {
3.          super.onResume();
4.          replaceFragment(mDeviceEventListFragment);
5.          mDeviceEventObserver.register(getApplicationContext());
6.          mInitializationTask = new InitializationTask(this);
7.          mInitializationTask.execute();
8.      }
```

Registering an event in MainActivity.java

2. Extend `DeviceEventsService.AbstractDeviceEventsChangeObserver` to detect an event.

```
1.  public class DeviceEventObserver extends
    DeviceEventsService.AbstractDeviceEventsChangeObserver {
2.      ObserverInterface observerInterface;
3.      public DeviceEventObserver(Handler handler, ObserverInterface observerInterface) {
4.          super(handler);
5.          this.observerInterface = observerInterface;
6.      }
7.      @Override
8.      public void onChange(DeviceEvent deviceEvent) {
9.          Log.i(MainActivity.TAG, Logger.build(deviceEvent));
10.         observerInterface.onChange(deviceEvent);
11.     }
12.     public interface ObserverInterface {
13.         void onChange(DeviceEvent deviceEvent);
14.     }
15. }
```

Extending an event in DeviceEventObserver.java

3. Unregister observer.

```
1.  @Override
2.      protected void onPause() {
3.          super.onPause();
4.          mDeviceEventObserver.unregister(getApplicationContext());
5.          mInitializationTask.cancel(true);
6.          mInitializationTask = null;
7.          if (mAlertDialog != null) {
8.              mAlertDialog.dismiss();
9.              mAlertDialog = null;
10.         }
11.     }
```

Unregistering an event in MainActivity.java

9 List of events supported by a device

This is the list of events which is supported by a device.

Alert Name
Cartridge Low
Toner Collection Unit Almost Full
Document Feeder Kit Low
Cartridge Very Low/Replace
Toner Collection Unit Full/Replace
Document Feeder Kit Very Low/Replace
Non-HP Supply Installed
This supply is missing or not seated.
Device Error
Internal clock error
Digital Send LDAP Server Error
Digital Send SMTP Server Error
Digital Send Server Error
Scanner Component Error
Remove Paper Jam
Close Drawers, Doors And Covers
Manually Feed
Output bin full
Tray Empty (Load)
Insufficient memory
41.3 Unexpected Paper Size
No job progress

10 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

11 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <http://www.hp.com/go/solutions> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.